



Tecnicatura Universitaria en Programación

Materia : Programación II

TPI : Food Store

**Sistema de Gestión de Pedidos de Comida
(Consola + JDBC)**

Integrantes:

Pablo Isaias Morinigo Lima

Gastón Puente

Juan Martinez

Suarez Giovanna

Fecha de entrega: 21/06/2026

Índice

Marco Teórico	3
Decisiones Técnicas y Arquitectura	4
Dificultades y Soluciones	7
Bibliografía/webgrafía	9
Links	9

Marco Teórico

En este proyecto se aplicaron conceptos fundamentales de la POO como encapsulamiento, herencia, polimorfismo y abstracción.

La clase abstracta Base fue utilizada para centralizar atributos comunes a todas las entidades del sistema, como id, eliminado y createdAt, evitando duplicación de código y favoreciendo la reutilización.

Además, se implementó la interfaz Calculable, utilizada por la clase Pedido para calcular el total de una compra a partir de sus detalles.

Gracias a la API JDBC que nos permite conectar aplicaciones con bases de datos relacionales, pudimos aprovechar sus funcionalidades para:

- Establecer conexiones con MySQL.
- Ejecutar operaciones CRUD.
- Recuperar información mediante consultas SQL.
- Gestionar transacciones para mantener la integridad de los datos.

Para evitar vulnerabilidades de inyección SQL se utilizaron objetos PreparedStatement.

Para la base relacional del proyecto, se crearon las siguientes tablas, respetando el modelo UML proporcionado por la cátedra.

- Categorías
- Productos
- Usuarios
- Pedidos
- Detalles de Pedido

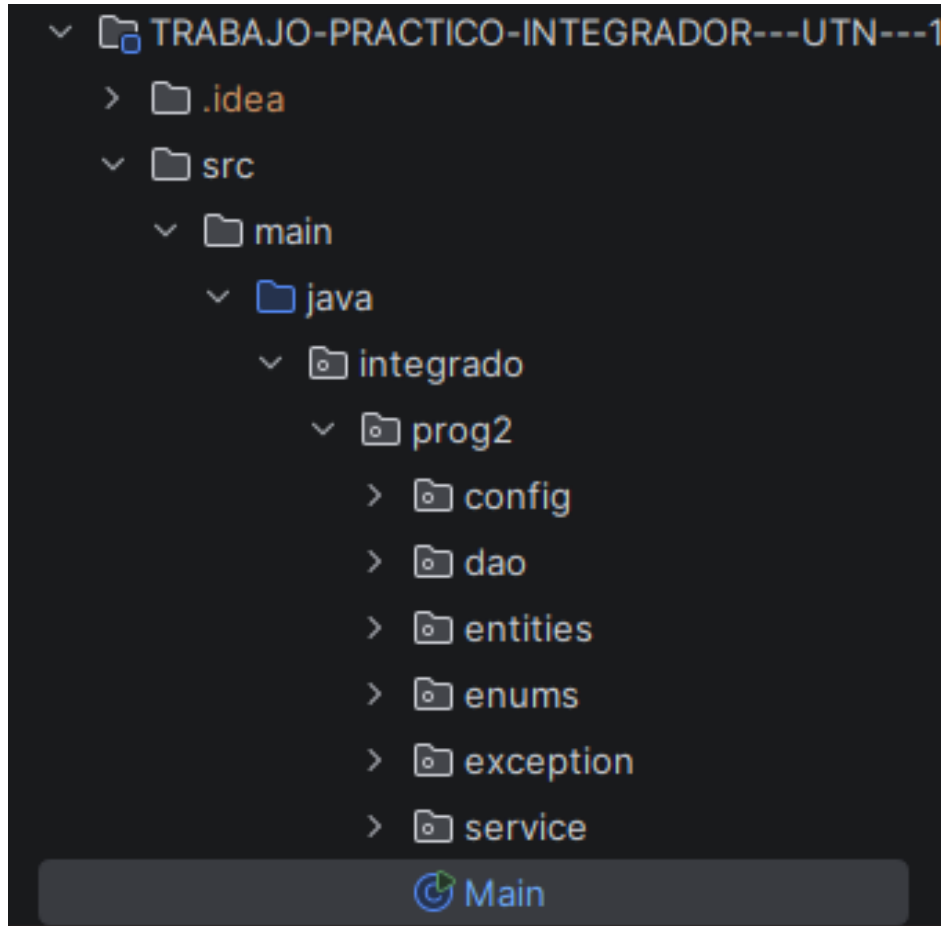
Gracias a el patrón DAO (Data Access Object) que nos permite separar la lógica de acceso a datos de la lógica de negocio pudimos lograr que :

- Las consultas SQL quedan centralizadas.
- El código es más mantenible.
- Se facilita la reutilización de componentes.
- Se mejora la organización del proyecto.

En el proyecto, cada entidad posee su correspondiente DAO encargado de interactuar con la base de datos.

Decisiones Técnicas y Arquitectura:

El proyecto fue organizado siguiendo una arquitectura en capas recomendada por la cátedra, resultando en :



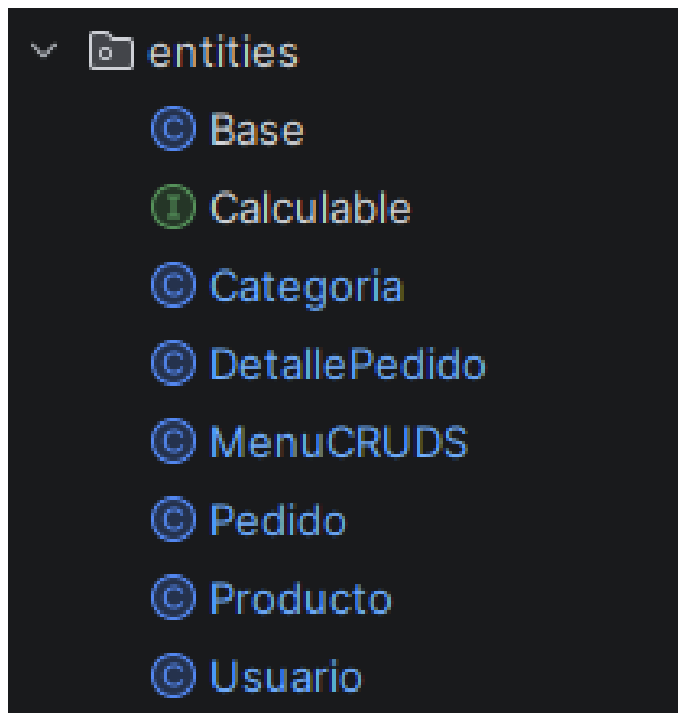
El proyecto fue desarrollado bajo el sistema de construcción Maven, el cual permite estandarizar la estructura del proyecto, administrar dependencias externas mediante el archivo pom.xml y automatizar tareas como la compilación, ejecución de pruebas y empaquetado. Las dependencias externas se descargan y almacenan en el repositorio local de Maven, evitando su inclusión manual en el proyecto.

Entities:

Contienen las clases del dominio del sistema:

- Categoria
- Producto
- Usuario
- Pedido
- DetallePedido
- MenuCRUDS

Representan la información y comportamiento principal de cada entidad.



DAO:

Implementan la persistencia mediante JDBC.

Son responsables de:

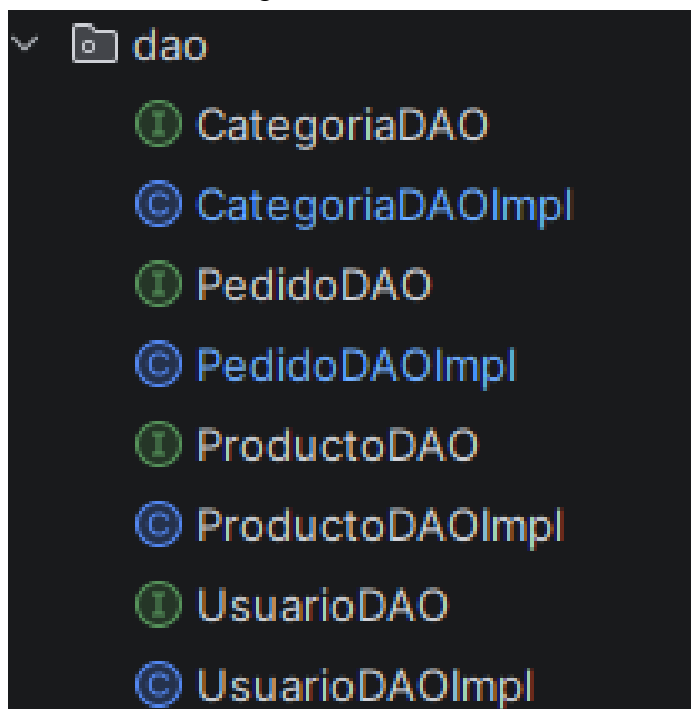
INSERT

UPDATE

DELETE lógico

SELECT

De esta forma ninguna consulta SQL se encuentra fuera de esta capa.



Services:

Contienen las reglas de negocio y validaciones.

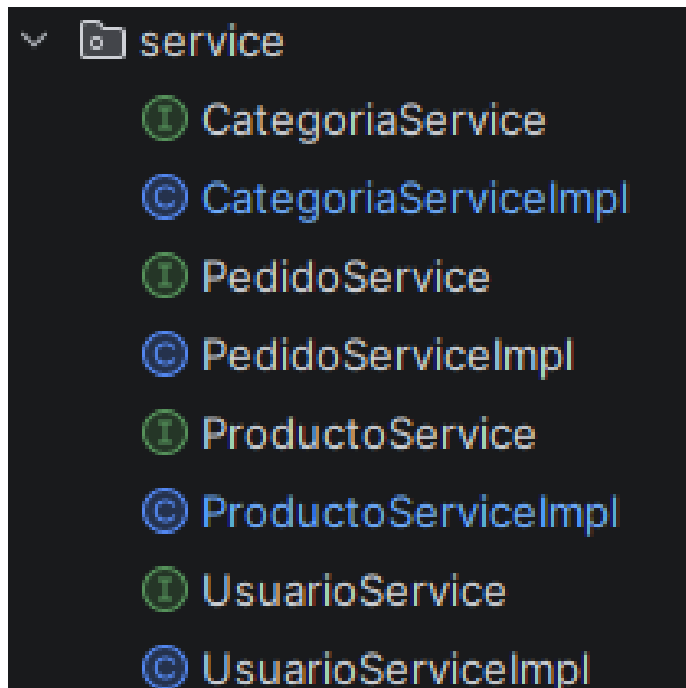
Por ejemplo:

Verificación de stock.

Validación de precios.

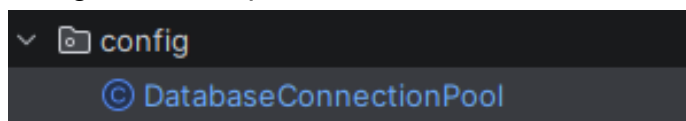
Control de estados de pedidos.

Cálculo de totales.



Config:

Centraliza la conexión a la base de datos mediante una clase ConexionDB y/o configuración en persistence.xml.



Main:

Gestionan la interacción con el usuario mediante consola utilizando Scanner.

Uso de Soft Delete:

Se decidió implementar bajas lógicas mediante el atributo eliminado.

Ventajas:

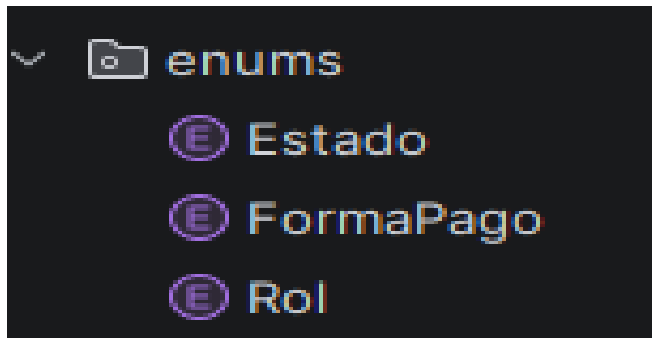
- Conservación del historial.
- Integridad referencial.
- Posibilidad de recuperación de información.

Uso de Enumeraciones:

Se utilizaron enums para representar valores fijos:

- Rol
- Estado
- FormaPago

Esto evita errores por cadenas de texto inválidas y mejora la legibilidad.



Dificultades y Soluciones:

Dificultad 1: Relaciones entre entidades

Uno de los principales desafíos fue modelar correctamente las relaciones entre categorías, productos, usuarios y pedidos respetando el UML propuesto.

Solución:

Se implementaron referencias entre objetos y claves foráneas en la base de datos, manteniendo la coherencia entre el modelo orientado a objetos y el modelo relacional.

Dificultad 2: Persistencia de Pedidos y Detalles

La creación de pedidos implica guardar información en múltiples tablas relacionadas.

Solución:

Se utilizaron transacciones JDBC para asegurar que todas las operaciones se ejecuten correctamente o se reviertan completamente en caso de error.

Dificultad 3: Manejo de Excepciones

Durante las pruebas surgieron errores relacionados con IDs inexistentes, correos duplicados y problemas de conexión.

Solución:

Se implementaron validaciones previas y excepciones personalizadas para mostrar mensajes claros al usuario sin interrumpir la ejecución del programa.

Dificultad 4: Trabajo Colaborativo con GitHub

La integración del trabajo realizado por distintos integrantes generó conflictos de versiones y diferencias entre ramas.

Solución:

Se utilizaron ramas individuales para cada integrante y posteriormente se realizaron procesos de merge para integrar los cambios al proyecto principal.

Bibliografía / Webgrafía:

- **Java Documentation:** <https://docs.oracle.com/en/java/>
- **JDBC Documentation:** <https://docs.oracle.com/javase/tutorial/jdbc/>
- **MySQL Documentation:** <https://dev.mysql.com/doc/>
- **Git Documentation:** <https://git-scm.com/doc>
- **IntelliJ IDEA Documentation:** <https://www.jetbrains.com/help/idea/>
- **Explicaciones :** Canal de Charly Cimino

Links

REPOSITORIO DE GIT:

<https://github.com/EIIsuss/TRABAJO-PRACTICO-INTEGRADOR---UTN---1PRO4---MENDOZA>